DTU Compute
Department of Applied Mathematics and Computer Science

Thesis title
Thesis subtitle

Master Thesis

Thesis title
Thesis subtitle

Master Thesis
Date, year

By
Author

Cover photo: Vibeke Hempler, 2012
Published by: DTU, Department of Applied Mathematics and Computer Science, Richard Petersens Plads, Building 324, 2800 Kgs. Lyngby Denmark
www.compute.dtu.dk
ISSN: [0000-0000] (electronic version)
ISBN: [000-00-0000-000-0] (electronic version)

ISSN: [0000-0000] (printed version)
ISBN: [000-00-0000-000-0] (printed version)

Approval

This thesis has been prepared over six months at the Section for Indoor Climate, Department of Civil Engineering, at the Technical University of Denmark, DTU, in partial fulfilment for the degree Master of Science in Engineering, MSc Eng.

It is assumed that the reader has a basic knowledge in the areas of statistics.

Author - s123456

.....

Signature

.....

Date

Abstract

There often appear cases where a particular marginal distribution in the generative process of a diffusion model is undesirable or better replaced with another distribution to suit a specific task. Scenarios such as fair generation or label shift are common examples of this need. Using Jeffrey's Rule, it is possible to transform this undesired distribution into a more suitable target distribution. In the context of distribution-guided diffusion, actually sampling from the posterior requires an accurate estimate of the clean data x_0 from a noisy sample x_t. Tweedie's formula is often employed for this purpose but its use of a single expected value, although good for simple classifier-guided diffusion, may not be best suited for a more complex case such as distribution-guidance. This thesis explores a Markov Chain Monte Carlo (MCMC) approach for sampling from the marginal distribution defined by Jeffreys Rule, investigating its performance within this distribution-guided diffusion setting. VERSION 2:

While diffusion models capture the data distributions they are trained on, the resulting learned marginals may require modification to satisfy constraints such as fairness or to account for label shift. Jeffrey's rule provides a principled mechanism for transforming an undesired marginal distribution into a desired target distribution.

Sampling from the resulting posterior during the diffusion process requires estimating the clean data x_0 from a noisy observation x_t. A common approach is to use Tweedie's formula, which provides an estimate of x_0 through the posterior mean. While effective in many settings, this mean-based approximation may be insufficient when the target distribution is complex or multi-modal, as can occur in distribution-guided diffusion models.

This thesis investigates the use of Markov Chain Monte Carlo (MCMC) methods to sample from the marginal distribution induced by Jeffrey's rule, evaluating their effectiveness within a distribution-guided diffusion framework.

Acknowledgements

Author, MSc Mathematical Modelling and Computation, DTU
Creator of this thesis template.

[Name], [Title], [affiliation]
[text]

[Name], [Title], [affiliation]
[text]

Contents

1 Introduction

Diffusion models, inspired by nonequilibrium statistical physics [1], are a popular paradigm of generative machine learning models that are prominently used for many different generation tasks such as image, video and audio synthesis as well as for scientific research tasks like 3D molecular generation and protein folding. [2] [3] [4] [5] [6] More recently they have also been adopted heavily in world simulations and physical modeling, which is used in areas such as robotics [7].

Training diffusion models means having them learn a distribution that matches the training data distribution as closely as possible. If the marginal distribution of the training data is somehow wrong, biased or doesn't reflect real world scenarios accurately then the generated samples of the trained model will reflect this in their output and give samples exhibiting this undesirable distribution. Even if the data distribution is fine there are still many situations where it is desirable to guide the generation process of the model toward some desired distribution, such as generating 50% male and 50% female images, or generating samples of particular colors.

Many different methods exist that allow for controlling the generation process. Incorporating the conditioning information into the training process [8] via classifier-guided diffusion or other similar conditional training approaches is one method but it requires training a task specific model along side large amounts of data, pairing the training data along the conditioning information. More flexible methods exist that allow for conditional sampling by directly working on the unconditional model. Twisted Diffusion Sampling [9] and Feynman-Kac steering [10] are recent examples of approaches that do not require task-specific training and allow for flexible control of the sampling process. Normally, TDS is done when the generation is conditioned on a fixed, single observation y, where y is defined through the likelyhood $p(y|x)$ [9], but combining it with a specific method for updating probability distributions, namely Jeffrey's rule, allows for steering toward a distribution instead. Feynman-Kac steering uses an arbitrary reward function to control the generation which naturally allows for steering toward a desired distribution.

Jeffrey's rule of conditioning is a generic way to update probability distributions in order to satisfy some given constraint. More specifically it says that the standard joint distribution

$$p(z_I, z_J) = p(z_I|z_J)p(z_J)$$

Can simply be rewritten as:

$$p^*(z_I, z_J) = p(z_I|z_J)p^*(z_J)$$

Where $p^*(z_J)$ is an adjusted marginal distribution of a subset of the features which had an undesirable marginal to begin with. This construction then leads to the formulation:

$$p^*(z_I, z_J) = \frac{p^*(z_J)}{p(z_J)}p(z_I, z_J)$$

Which says that the new joint distribution is an importance-weighted version of the original p. Combining Jeffrey's conditioning with the above mentioned methods for steering the sampling, it is possible to condition the sampling process on an arbitrary distribution. (CHECK? IS IT TRUE? FOR ANY ARBTRIARY DISTRIBUTION?)

This thesis explores using Jeffrey's rule CITATION needed(Jeffrey PDF from Jes? OR perhaps dig deeper into the refernces listed on there.) in combination with the algorithms for conditioning the sampling process of a diffusion model in order to steer the generation towards a desired target distribution. These methods will be evaluated and compared in terms of their sampling efficiency, distributional fidelity, and ability to maintain sample diversity while adhering to external constraints. (DOUBLE CHECK LAST SENTENCE, WHAT TO COMPARE?)

2 Background

This chapter gives the reader background into the theory that this thesis is based on. First, score-based diffusion models are explained via stochastic differential equations, which provides the core framework for the diffusion model implementation of the thesis. Secondly, the core workings of Jeffrey's rule of conditioning are layed down and how it can be used to modify a joint distribution to include a more desirable marginal. Finally, the essentials of some algorithms for doing distribution-guided generation are described.

2.1 Diffusion Models

Diffusion models are probabilistic generative models that operate through two distinct processes, a forward process and a reverse process. The forward process slowly corrupts the initial data into noise. Then, the reverse process learns to reverse this corrupted data in order to generate a clean sample. Diffusion models were first introduced in [1], taking inspiration from nonequilibrium thermodynamics. The forward and reverse processes were formulated as discrete Markov chains, where at each time step t, the state of the noisy data only depends on the state at the previous time step $t-1$. Formally the forward process is defined as:

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1}) \text{ where}$$
$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

Where the β_t form a variance schedule $\{\beta_t \in (0,1)\}_{t=1}^{T}$ that controls how much noise is added at each step, ensuring that at the end the data has become pure Gaussian noise. The reverse process is then formulated as:

$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$$
$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

The goal was then to learn the transition distributions of the reverse process, which would finally yield clean images. Initially, the reverse transitions were parametrized by a neural network where training was done by maximizing the lower bound of the model log likelihood. Subsequent work showed that instead of learning the log likelihood it was possible to learn the score of the perturbed data distribution, which is the gradient of the log of the distribution and points to where higher density regions of clean data lie [11]. Furthermore, it was shown that learning the log likelihood of the reverse transitions, which is in practice done by learning the Evidence Lower Bound (ELBO), can be mathematically reformulated as learning to predict the noise added during the forward process.[2]. This learning objective was proved to be functionally equivalent to learning the score. Diffusion models that learn the score, either directly or implicitly through this reformulation of the ELBO are called score-based diffusion models.

Two very successful types of score-based diffusion models are Denoising diffusion propabilistic models (DDPM) [2] and Score matching with Langevin dynamics (SMLD) [11][1]. A

[1] While SMLD is today categorized as a core diffusion model, it was originally introduced purely as a score-matching technique. It was grouped into the diffusion family only after subsequent work proved its mathematical equivalency to DDPM and unified both methods under the continuous-time SDE framework.

breakthrough in diffusion model research occurred when these two score-based methods were unified and shown to be different discretizations of Stochastic Differential Equations (SDEs) [12], a perspective that generalized these frameworks into continuous-time diffusion processes.

2.2 Score-based Diffusion Models via SDEs

Score-based diffusion models via SDEs use an infinite number of noise scales to perturb the data such that it evolves according to an SDE. The forward diffusion process then becomes $\{\mathbf{x}(t)\}_{t=0}^{T}$, indexed by the continuous time variable $t \in [0, T]$ instead of being made up of a discrete, finite number of noise scales. This diffusion process is such that when t is equal to 0 it matches the training data distribution and when $t = T$ it matches some given prior distribution which is usually Gaussian noise with a fixed mean and variance. The reverse process is then modelled by the reverse-time SDE of the forward process, which is itself a diffusion process [13]. The forward diffusion process can be formulated as the solution to the Itô SDE [14].

$$\mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t + g(t)\mathrm{d}\mathbf{w} \tag{2.1}$$

where $f(x, t) : \mathbb{R}^d \to \mathbb{R}^d$ is a vector valued function which is named the *drift* coefficient of $\mathbf{x}(t)$, $g : \mathbb{R} \to \mathbb{R}$ is called the *diffusion* coefficient of $\mathbf{x}(t)$ and $\mathbf{w}$ is the Wiener process. The solution is a continuous stochastic process that starts equal to the data distribution at $t = 0$ and ends as noise when $t = T$. At time step t the process can be described by the probability density $p_t(\mathbf{x})$ and the transition from a previous state to the next can be described by the conditional density $p(\mathbf{x}(t)|\mathbf{x}(s))$ where $0 \leq s < t \leq T$.

The reverse-process can be modelled by the reverse-time SDE:

$$\mathrm{d}\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x})]\mathrm{d}t + g(t)\mathrm{d}\bar{\mathbf{w}} \tag{2.2}$$

The solution to this SDE is then a stochastic process such that when simulated yields samples at $t = 0$. The only obstacle is that the score, $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, is generally intractable and thus has to be estimated somehow. Score-matching [15] allows for predicting the score via a score-based model, $\mathbf{s}_\theta(\mathbf{x}, t)$, often a neural network which is trained via:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathbb{E}_t \left\{ \lambda(t) \mathbb{E}_{\mathbf{x}(0)} \mathbb{E}_{\mathbf{x}(t)|\mathbf{x}(0)} \left[\left\| \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t), t) - \nabla_{\mathbf{x}(t)} \log p(\mathbf{x}(t)|\mathbf{x}(0)) \right\|_2^2 \right] \right\} \tag{2.3}$$

Where $\lambda : [0, T] \to \mathbb{R}^+$ is positive weighting function and we sample t uniformly from $[0, T]$. Note that for the score, the conditional density $p(x(t)|x(0))$ is used, instead of the marginal of $x(t)$, this is again because the marginal is generally hard to compute and a result from [16] shows that the scores are equal for the two distributions up to a constant. This method of score matching is called Denoising Score Matching since it uses the conditional of the noisy sample given the clean one to calculate the score. Once we have access to the score estimation, sampling can be done by using a general SDE solver and simulating the answer to the reverse SDE.

2.3 Jeffrey's rule of Conditioning

Jeffrey's rule of conditioning is a general method for changing a probability distribution to satisfy some given constraint. [17] The rule can be defined for both discrete and continuous distributions, with sums being replaced by Lebesque integrals and ratios by Radon-Nikodym derivatives accordingly. The discrete case is stated here, since it is a bit simpler and less verbose, but analogous results hold for the continuous case. Let

$\mathcal{Z} = \mathcal{Z}_1 \times \mathcal{Z}_2 \times \ldots \times \mathcal{Z}_d$ be the Cartesian product of countable sets. We consider a probability distribution over $\mathcal{Z}$ with full support, meaning every event has a non-zero probability of occurring. This allows for writing the probability distribution as a strictly positive probability mass function $p : \mathcal{Z} \to (0, 1]$. Now, we assume that there is some subset of the d variables, denoted as $J \subset \{1, \ldots, d\}$, whose marginal distribution, $p(z_J)$, we are unhappy with. This marginal density of the set J is the joint marginal density of all the elements in J. The goal is to replace this with a desired marginal $p^*(z_J)$. Denote the set of variables we are happy with as the complement of J, that is $I = \{1, \ldots, d\} \setminus J$, then the classic factorization of a joint density is:

$$p(z_I, z_J) = p(z_I \mid z_J)p(z_J) \tag{2.4}$$

The *Jeffrey updated joint density* is then defined by simply replacing the undesired marginal with the desired one,

$$p^*(z_I, z_J) = p(z_I \mid z_J)p^*(z_J) \tag{2.5}$$

This new distribution p^* has, by construction, the correct marginal for z_J and likewise the conditional density of the desirable variables given the desirable ones is preserved, a property often called ”rigidity” or the ”J condition”. A noteworthy property of the construction of p^* is that the marginal of z_I,changes according to

$$p^*(z_I) = \sum_{z_J \in \mathcal{Z}_J} p(z_I \mid z_J)p^*(z_J) \tag{2.6}$$

Furthermore, p^* is an *importance-weighted* version of p since by the definition in (2.5) and the factorization in (2.4), we obtain

$$p^*(z_I, z_J) = p(z_I \mid z_J)p^*(z_J) = \frac{p^*(z_J)}{p(z_J)} p(z_I, z_J) \tag{2.7}$$

Theorem 1 in [17] then states a direct consequence of this property, that all conditionals and marginals of p^* can be expressed as reweighted versions of the original distribution p. Another interesting characteristic of this formulation of p^* is, that it is the distribution closest to p that has the desired marginal for z_J. No other distribution is more like p while having the correct marginal. Jeffrey’s rule therefore provides a principled way to replace an undesired marginal distribution wth a desired target marginal while preserving the corresponding structure. How exactly this update is implemented in practice depends on the problem setting and specific sampling method.

2.4 Twisted Diffusion Sampling

Twisted Diffusion Sampling (TDS) is a sequential Monte Carlo (SMC) algorithm that allows for flexible conditional generation of samples [9], without needing a conditionally trained model specific to only a single conditional generation task. SMC algorithms are a general way of sampling from a sequence of distributions on variables $x^{0:T}$, terminating at some final target distribution. They work by generating an ensemble of K *particles* $\{x_k^t\}_{k=1}^K$ at timestep t of an iterative procedure which has T steps in total, often denoted as starting at timestep T and going down to 0. The main components of SMC are, weighting functions denoted by $w_T(x^T)$, $\{w_t(x^t, x^{t+1})\}_{t=0}^{T-1}$ and proposals, which are distributions and are denoted by $r^T(x^T)$, $\{r_t(x^t|x^{t+1})\}_{t=0}^{T-1}$, that are used at each step of the procedure. More definitively, the general recipe for an SMC algorithm is the following:
At initial timestep T; K particles are drawn $x_k^T \sim r_T(x^T)$ and the weights are set to

$w_k^T := w_T(x_k^T)$, then repeat sequentially for $T-1, \ldots, 0$:

$$\begin{aligned}
&\text{resample: } \{x_k^{t+1:T}\}_{k=1}^K \sim \text{Multinomial}(\{x_k^{t+1:T}\}_{k=1}^K; \{w_k^{t+1}\}_{k=1}^K) \\
&\text{propose: } x_k^t \sim r_t(x^t | x^{t+1}), k = 1, \ldots, K \\
&\text{weight: } w_k^t := w_t(x_k^t, x_k^{t+1}), k = 1, \ldots, K
\end{aligned}$$

At each timestep t the weighting functions and proposals form an intermediate target distribution [9] denoted by ν_t and defined as:

$$\nu_t(x^{0:T}) := \frac{1}{\mathcal{L}_t} \left[r_T(x^T) \prod_{t'=0}^{T-1} r_{t'}(x^{t'} \mid x^{t'+1}) \right] \left[w_T(x^T) \prod_{t'=t}^{T-1} w_{t'}(x^{t'}, x^{t'+1}) \right] \tag{2.8}$$

With *normalization* constant $\mathcal{L}_t$. These intermediate distributions are the joint probability distributions of all the variables $x^{0:T}$ with the path from T to t already having been weighted. They are the product of all the proposals, which are distributions, and the weights from t to T. The constant $\mathcal{L}_t$ is so that they integrate to 1 and form a valid probability distribution. Marginalizing the variables $x^{1:T}$ out of the final distribution $\nu_0(x^{0:T})$, yields $\nu_0(x^0)$ which is the final target distribution of interest. It is the distribution of the clean sample x^0 induced by the fully weighted distribution over trajectories $x^{0:T}$.

For conditional generation the standard setup is that there is some observation y related to the clean data variable x^0 through the likelihood $p(y|x^0)$ which may be given by a classifier, a learned scoring function, or some problem-specific model. This, together with the diffusion model $p_\theta(x^{0:T})$ defines a joint distribution $p_\theta(x^{0:T}, y) = p_\theta(x^{0:T})\, p(y \mid x^0)$ where y depends only on x^0, since only the clean data x^0 is needed for conditioning and not the noisy x^t. The goal is then to sample from $p_\theta(x^0 \mid y)$. By using the Markov chain properties of diffusion models and including the rest of the variables $x^{1:T}$ it is possible to rewrite the conditional as:

$$p_\theta(x^{0:T} \mid y) = \frac{p_\theta(x^{0:T}, y)}{p_\theta(y)} = \frac{1}{p_\theta(y)} \left[p(x^T) \prod_{t=0}^{T-1} p_\theta(x^t \mid x^{t+1}) \right] p_{y|x^0}(y \mid x^0) \tag{2.9}$$

This factorization naturally fits into the SMC framework and matches the definition of the intermediate distributions nicely.

By choosing appropriate proposals and weights, the terminal distribution of an SMC algorithm can be made to be $p_\theta(x^{0:T} \mid y)$, and thus the final target of interest, $v_0(x^0)$ becomes equal to $p_\theta(x^0 \mid y)$. Now, this target $p_\theta(x^0 \mid y)$, is generally intractable to compute analytically since it requires marginalizing the variables $x^{1:T}$ which in turn requires evaluating an intractable, high-dimensional integral over the highly non-linear transition distributions parameterized by a neural network, i.e. the $p_\theta(x^t \mid x^{t+1})$. Here, a fundamental property of SMC algorithms comes into play: The weighted particles form a discrete approximation to $\nu_t(x^t)$ at each timestep t:

$$\left(\sum_{k'=1}^K w_{k'}^t \right)^{-1} \sum_{k=1}^K w_k^t \delta_{x_k^t}(x^t) \tag{2.10}$$

Where δ is a Dirac measure. Under standard assumptions, this empirical approximation converges setwise to $\nu_t(x^t)$ as the number of particles $K \to \infty$ (Proposition 11.4 in [18], Appendix A.5 in [9]). Consequently, this SMC approximation can be made arbitrarily accurate, and therefore provides an arbitrarily accurate approximation of the desired conditional distribution.

As mentioned above, the proposals and weights have to be chosen in a certain way so that the conditional target can be approximated by an SMC method. One such way to choose the proposals and weights is the naive way of choosing them to be the marginal and transition distributions of (2.9). That is, by setting $r_T(x^T) = p(x^T)$ and $r_t(x^t \mid x^{t+1}) = p_\theta(x^t \mid x^{t+1})$ for $1 \leq t \leq T$, and the weighting functions as $w_t(x^t, x^{t+1}) = 1$ for $1 \leq t \leq T$ with $w_0(x^0, x^1) = p_{y|x^0}(y \mid x^0)$ the final target distribution becomes $\nu_0 = p_\theta(x^{0:T} \mid y)$ with normalizing constant $\mathcal{L}_t = p_\theta(y)$. Which then gives that $\mathbb{P}_K(x^0 \mid y) := (\sum_{k=1}^K w_k^0)^{-1} \sum_{k=1}^K w_k^0 \delta_{x_k^0}(x^0)$ becomes an approximation to $p_\theta(x^0 \mid y)$. This is equivalent to simulating the entire unconditional diffusion process and then simply weighting the trajectories according to how likely the observation y is given the final sample x_0. If the conditional distribution $p_\theta(x^0 \mid y)$ differs substantially from the marginal $p_\theta(x^0)$, then samples drawn from the unconditional diffusion proposal will typically have very small likelihood $p(y \mid x^0)$. As a result, their importance weights become highly uneven, with only a small fraction of particles receiving non-negligible weight. The result in [19] formalizes this and says that the number of particles required for accurate estimation of $p_\theta(x^{0:T} \mid y)$ is exponential in $\mathrm{KL}[p_\theta(x^{0:T} \mid y) \parallel p_\theta(x^{0:T})]$. Meaning that if the conditional and marginal are too different, the KL divergence becomes large and the number of particles needed for effective sampling becomes very high.

To reduce the number of particles needed for a good approximation of the final target distribution, a method called *twisting* can be used. [20] [21] [22]. Twisting uses a sequence of *twisting functions* that alter the naive proposals and weights such that the intermediate distributions become closer to the final target. There exists optimal twisting proposals, denoted as r_t^* that are defined by multiplying the naive proposals by $p_\theta(y \mid x^t)$, that is:

$$r_T^*(x^T) \propto p(x^T)\, p_\theta(y \mid x^T) \text{ and } r_t^*(x^t \mid x^{t+1}) \propto p_\theta(x^t \mid x^{t+1})\, p_\theta(y \mid x^t), \quad 0 \leq t < T. \tag{2.11}$$

By using these proposals an SMC sampler draws exact samples from $p_\theta(x^{0:T} \mid y)$ since by Bayes rule we get:

$$r_T^*(x^T) = p_\theta(x^T \mid y) \quad \text{and} \quad r_t^*(x^t \mid x^{t+1}) = p_\theta(x^t \mid x^{t+1}, y), \quad 0 \leq t < T. \tag{2.12}$$

Sampling from $x^T \sim r_T^*(x^T)$ and $x^t \sim r^*(x^t \mid x^{t+1})$ for $t = T-1, \ldots, 0$ is then equal to sampling from $p_\theta(x^{0:T} \mid y)$ by the chain rule of probability and the factorization in (2.9). The law of total probability then gives that the final sample obtained is $x^0 \sim p_\theta(x^0 \mid y)$.

This poises one problem, generally the distributions $p_\theta(y \mid x^t)$ are not analytically tractable. Writing $p_\theta(y \mid x^t) = \int p(y|x^0) p_\theta(x^0 \mid x^t)$, one can see that evaluating this quantity requires access to $p_\theta(x^0 \mid x^t)$. However, this distribution is itself intractable, as it involves marginalizing over the intermediate variables $x^{1:t-1}$ from the joint distribution $p_\theta(x^{0:t-1} \mid x^t)$, which is intractable. Sampling from these twisted proposals is therefore not generally easy to do. Instead, TDS approximates the twisting functions using the denoising neural network approximations:

$$\tilde{p}_\theta(y \mid x^t) := p_{y|x^0}(y \mid \hat{x}_\theta(x^t)) \approx p_\theta(y \mid x^t), \tag{2.13}$$

and uses this approximation to define the twisted proposals as:

$$\tilde{p}_\theta(x^t \mid x^{t+1}, y) := \mathcal{N}\left(x^t; x^{t+1} + \sigma^2 s_\theta(x^{t+1}, y), \tilde{\sigma}^2\right), \tag{2.14}$$

$$\text{where} \quad s_\theta(x^{t+1}, y) := s_\theta(x^{t+1}) + \nabla_{x^{t+1}} \log \tilde{p}_\theta(y \mid x^{t+1}) \tag{2.15}$$

and the twisted weights as:

$$w_t(x^t, x^{t+1}) := \frac{p_\theta(x^t \mid x^{t+1})\tilde{p}_\theta(y \mid x^t)}{\tilde{p}_\theta(y \mid x^{t+1})\tilde{p}_\theta(x^t \mid x^{t+1}, y)}, \qquad t = 0, \ldots, T-1 \tag{2.16}$$

The approximation in (2.13) is tractable, it only needs a single call to the neural network to give an estimate of x^0 and furthermore it doesnt require evaluating a huge integral, since the approximation evaluates y against the single most likely expected value of x^0 given x^t instead of the entire distribution of possible x^0. The neural network is trained to approximate the expected value $\mathbb{E}[x^0|x^t]$, either directly or implicitly through the score or noise, meaning that the single estimate of x^0 becomes increasingly accurate as t approaches 0, and at $t = 0$, it is easy to make the approximation an equality, by choosing $\hat{x}_\theta(x^0) = x^0$, yielding $\tilde{p}_\theta(y \mid x^0) = p(y \mid x^0)$. The twisted proposals use an approximation of the conditional score in definition (2.15), that is $s_\theta(x^{t+1}, y) \approx \nabla_{x^{t+1}} \log p_\theta(x^{t+1} \mid y)$, which is made up of approximations of the unconditional score and the conditional score of y given x^{t+1}. Alongside this, there is the proposal variance $\tilde{\sigma}^2$, which can require computationally expensive second-derivative calculations to optimize fully. Therefore, for simplicity, it is standard practice to set it equal to the unconditional transition variance σ^2. The weighting functions are non-trivial here, because in general the twisted proposals in (2.14) will not match the optimal twisted proposals from (2.12), and so they are needed for ensuring the terminal distribution converges to the desired target. By replacing the proposals and weights in (2.8) with their twisted counterparts in (2.14) and (2.16) and then simplifying, the intermediate distributions become:

$$\nu_t(x^{0:T}) \propto p_\theta(x^{0:T} \mid y) \left[\frac{\tilde{p}_\theta(y \mid x^t)}{p_\theta(y \mid x^t)} \prod_{t'=0}^{t-1} \frac{\tilde{p}_\theta(x^{t'} \mid x^{t'+1}, y)}{p_\theta(x^{t'} \mid x^{t'+1}, y)} \right] \tag{2.17}$$

When $t = 0$, we have that $\tilde{p}_\theta(y \mid x^0) = p_\theta(y \mid x^0)$ by construction, and so (2.17) becomes $\nu_0(x^{0:T}) = p_\theta(x^{0:T} \mid y)$, as desired. The discrete approximations then arbitrarily approximate $p_\theta(x^0 \mid y)$ understandard conditions (POTENTIALLY LIST THEM? IN APPENDIX OR?). The Twisted Diffusion Sampler (TDS) algorithm as written in [9], is given in (1)

Algorithm 1: Twisted Diffusion Sampler (TDS)

for $k = 1 : K$ **do**
  $x_k^T \sim p(x^T), \quad w_k \leftarrow \tilde{p}_k^T = \tilde{p}_\theta(y \mid x_k^T)$; `// initial proposal and weight`
for $t = T-1, \cdots, 0$ **do**
  $\{x_k^{t+1}, \tilde{p}_k^{t+1}\}_{k=1}^K \sim$ Multinomial $\left(\{x_k^{t+1}, \tilde{p}_k^{t+1}\}_{k=1}^K; \{w_k\}_{k=1}^K\right)$; `// resample`
  for $k = 1 : K$ **do**
    $\tilde{s}_k = (\hat{x}_\theta(x_k^{t+1}) - x_k^{t+1})/t\sigma^2 + \nabla_{x_k^{t+1}} \log \tilde{p}_k^{t+1}$; `// conditional score approx.`
    $x_k^t \sim \tilde{p}_\theta(\cdot \mid x_k^{t+1}, y) := \mathcal{N}\left(x_k^{t+1} + \sigma^2 \tilde{s}_k, \tilde{\sigma}^2\right)$; `// proposal`
    $\tilde{p}_k^t \leftarrow \tilde{p}_\theta(y \mid x_k^t)$; `// twisting function`
    $w_k \leftarrow p_\theta(x_k^t \mid x_k^{t+1})\tilde{p}_k^t / [\tilde{p}_\theta(x_k^t \mid x_k^{t+1}, y)\tilde{p}_k^{t+1}]$; `// weight`

2.4.1 From conditional TDS to Jeffrey-guided TDS

The formulation and motivation of the Twisted Diffusion Sampler, given above, assumes a standard conditional generation setup. Where, the diffusion model parametrizes a distribution $p_\theta(x^{0:T})$, and another assumed to be known model $p(y \mid x^0)$ predicts information y given x^0. Together they define a joint distribution $p_\theta(x^{0:T}, y) = p_\theta(x^{0:T})p(y \mid x^0)$, with the goal then being to sample from the conditional $p_\theta(x^0 \mid y)$. In this thesis, the setting is different. Rather than conditioning on a single fixed value of y, the goal is distribution-guided

generation: to generate samples whose implied attribute marginal matches a desired target marginal $p^*(y)$.

Concretely, writing the joint as $p_\theta(x^{0:T}, y) = p_\theta(x^{0:T} \mid y)p_\theta(y)$, we treat the original model-induced marginal $p_\theta(y)$ as undesirable and replace it with $p^*(y)$. Applying Jeffrey's rule gives

$$p^*_\theta(x^{0:T}, y) = p_\theta(x^{0:T} \mid y)\, p^*(y) = \frac{p^*(y)}{p_\theta(y)}\, p_\theta(x^{0:T}, y). \tag{2.18}$$

Here

$$p_\theta(y) = \int p_\theta(x^{0:T})\, p(y \mid x^0)\, dx^{0:T},$$

and we assume $p(y) > 0$ for all y in the domain of interest; that is, the marginal $p(y)$ has full support over the relevant set of y values. By marginalizing y out of the updated joint model, we get,

$$p^*_\theta(x^{0:T}) = \sum_y p^*_\theta(x^{0:T}, y) = p_\theta(x^{0:T}) \sum_y \frac{p^*(y)}{p_\theta(y)} p(y \mid x^0) \tag{2.19}$$

Where the last step follows from the factorization in (2.18) and the fact that $p_\theta(x^{0:T}, y) = p_\theta(x^{0:T})p(y \mid x^0)$. Equation (2.19) is the new TDS target and the goal is then to sample from $p^*_\theta(x^0)$, after marginalizing out $x^{1:T}$. Particularly this is written as:

$$p^*_\theta(x^0) = \int p^*_\theta(x^{0:T}) dx^{1:T} \tag{2.20}$$

Denoting the guidance term in (2.19) by $G(x^0) := \sum_y \frac{p^*(y)}{p_\theta(y)} p(y \mid x^0)$, the target changes from $p_\theta(x^{0:T} \mid y)$ in the fixed conditional TDS case to $p^*_\theta(x^{0:T}) = p_\theta(x^{0:T})G(x^0)$. The tractable twisting functions are then similar, except that the approximation of $\tilde{p}_\theta(y \mid x^t)$ is replaced by

$$\tilde{G}_t(x^t) := \sum_y \frac{p^*(y)}{p_\theta(y)} p(y \mid \hat{x}_\theta(x^t)) \tag{2.21}$$

and since the distributions are no longer conditioned on fixed y the score approximation, proposals and weights in 2.14 2.15 2.16 become,

$$\tilde{p}^*_\theta(x^t \mid x^{t+1}) := \mathcal{N}\big(x^t;\ x^{t+1} + \sigma^2 s^*_\theta(x^{t+1}), \tilde{\sigma}^2\big)\,. \tag{2.22}$$

$$s^*_\theta(x^{t+1}) := s_\theta(x^{t+1}) + \nabla_{x^{t+1}} \log \tilde{G}_{t+1}(x^{t+1}) \ \approx\ \nabla_{x^{t+1}} \log p^*_\theta(x^{t+1}). \tag{2.23}$$

$$w^*_t(x^t, x^{t+1}) := \frac{p_\theta(x^t \mid x^{t+1})\, \tilde{G}_t(x^t)}{\tilde{G}_{t+1}(x^{t+1})\, \tilde{p}^*_\theta(x^t \mid x^{t+1})} \tag{2.24}$$

with $w^*_T = \tilde{G}_T(x^T)$

3 Methodology

This chapter establishes the methods used for the experiments in the thesis. The experiments are rigorously described and defined, what exactly was done and why it was done and the expected results. The first experiement was a simple bivariate toy experiment,

Bibliography

[1] Jascha Sohl-Dickstein et al. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. 2015. arXiv: 1503.03585 [cs.LG]. URL: https://arxiv.org/abs/1503.03585.

[2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239 [cs.LG]. URL: https://arxiv.org/abs/2006.11239.

[3] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv: 2112.10752 [cs.CV]. URL: https://arxiv.org/abs/2112.10752.

[4] Jonathan Ho et al. *Video Diffusion Models*. 2022. arXiv: 2204.03458 [cs.CV]. URL: https://arxiv.org/abs/2204.03458.

[5] Zhifeng Kong et al. *DiffWave: A Versatile Diffusion Model for Audio Synthesis*. 2021. arXiv: 2009.09761 [eess.AS]. URL: https://arxiv.org/abs/2009.09761.

[6] Minkai Xu et al. *GeoDiff: a Geometric Diffusion Model for Molecular Conformation Generation*. 2022. arXiv: 2203.02923 [cs.LG]. URL: https://arxiv.org/abs/2203.02923.

[7] Ze Yang et al. *UniSim: A Neural Closed-Loop Sensor Simulator*. 2023. arXiv: 2308.01898 [cs.CV]. URL: https://arxiv.org/abs/2308.01898.

[8] Prafulla Dhariwal and Alex Nichol. *Diffusion Models Beat GANs on Image Synthesis*. 2021. arXiv: 2105.05233 [cs.LG]. URL: https://arxiv.org/abs/2105.05233.

[9] Luhuan Wu et al. *Practical and Asymptotically Exact Conditional Sampling in Diffusion Models*. 2024. arXiv: 2306.17775 [stat.ML]. URL: https://arxiv.org/abs/2306.17775.

[10] Raghav Singhal et al. "A General Framework for Inference-time Scaling and Steering of Diffusion Models". In: *Forty-second International Conference on Machine Learning*. 2025. URL: https://openreview.net/forum?id=Jp988ELppQ.

[11] Yang Song and Stefano Ermon. *Generative Modeling by Estimating Gradients of the Data Distribution*. 2020. arXiv: 1907.05600 [cs.LG]. URL: https://arxiv.org/abs/1907.05600.

[12] Yang Song et al. *Score-Based Generative Modeling through Stochastic Differential Equations*. 2021. arXiv: 2011.13456 [cs.LG]. URL: https://arxiv.org/abs/2011.13456.

[13] Brian D.O. Anderson. "Reverse-time diffusion equation models". In: *Stochastic Processes and their Applications* 12.3 (1982), pp. 313–326. ISSN: 0304-4149. DOI: https://doi.org/10.1016/0304-4149(82)90051-5. URL: https://www.sciencedirect.com/science/article/pii/0304414982900515.

[14] Bernt Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. 6th. Universitext. Berlin Heidelberg: Springer-Verlag, 2010. ISBN: 978-3-642-14394-6. DOI: 10.1007/978-3-642-14394-6.

[15] Aapo Hyvärinen. "Estimation of Non-Normalized Statistical Models by Score Matching". In: *J. Mach. Learn. Res.* 6 (Dec. 2005), pp. 695–709. ISSN: 1532-4435.

[16] Pascal Vincent. "A Connection Between Score Matching and Denoising Autoencoders". In: *Neural Computation* 23.7 (2011), pp. 1661–1674. DOI: 10.1162/NECO_a_00142.

[17] Pierre-Alexandre Mattei. "Jeffrey's rule". Preprint. Under review. Inria.

[18] Nicolas Chopin and Omiros Papaspiliopoulos. *An Introduction to Sequential Monte Carlo*. Springer Series in Statistics. Cham: Springer, 2020. ISBN: 978-3-030-47845-2. DOI: 10.1007/978-3-030-47845-2.

[19] Sourav Chatterjee and Persi Diaconis. *The sample size required in importance sampling*. 2017. arXiv: 1511.01437 [math.PR]. URL: https://arxiv.org/abs/1511.01437.

[20] Pieralberto Guarniero, Adam M. Johansen, and Anthony Lee. *The iterated auxiliary particle filter*. 2016. arXiv: 1511.06286 [stat.CO]. URL: https://arxiv.org/abs/1511.06286.

[21] Jeremy Heng et al. "Controlled sequential Monte Carlo". In: *The Annals of Statistics* 48.5 (Oct. 2020). ISSN: 0090-5364. DOI: 10.1214/19-aos1914. URL: http://dx.doi.org/10.1214/19-AOS1914.

[22] Christian A. Naesseth, Fredrik Lindsten, and Thomas B. Schön. *Elements of Sequential Monte Carlo*. 2024. arXiv: 1903.04797 [stat.ML]. URL: https://arxiv.org/abs/1903.04797.

A Title

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Technical
University of
Denmark

Richard Petersens Plads, Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk